

Họ và tên: Đoàn Anh Quân

MSSV: 23110111

BÀI THỰC HÀNH DATA MINING TUẦN 5

Cài đặt và thực thi code thuật toán Vertical Apriori:

- Import các thư viện và load dữ liệu:

```
import numpy as np
import pandas as pd

df = pd.read_csv(r"D:\data_mining\tuan5\data.csv")
print(df)
```

✓ 0.0s

	TID	Itemset
0	1	Wine, Chips, Bread, Butter, Milk, Apple
1	2	Wine, Bread, Butter, Milk
2	3	Bread, Butter, Milk
3	4	Chips, Apple
4	5	Wine, Chips, Bread, Butter, Milk, Apple
5	6	Wine, Chips, Milk
6	7	Wine, Chips, Bread, Butter, Apple
7	8	Wine, Chips, Milk
8	9	Wine, Bread, Apple
9	10	Wine, Bread, Butter, Milk
10	11	Chips, Bread, Butter, Apple
11	12	Wine, Butter, Milk, Apple
12	13	Wine, Chips, Bread, Butter, Milk
13	14	Wine, Bread, Milk, Apple
14	15	Wine, Bread, Butter, Milk, Apple
15	16	Wine, Chips, Bread, Butter, Milk, Apple
16	17	Chips, Bread, Butter, Milk, Apple
17	18	Chips, Butter, Milk, Apple
18	19	Wine, Chips, Bread, Butter, Milk, Apple
19	20	Wine, Bread, Butter, Milk, Apple
20	21	Wine, Chips, Bread, Milk, Apple
21	22	Chips

- Code lại thuật toán Apriori theo mã giả, trong đó thay min_support thành min_count:
- Hàm has_infrequent_subset:

```
from itertools import combinations

def has_infrequent_subset(c, L_k_minus_1):
    subsets = combinations(c, len(c) - 1)

    for s in subsets:
        if s not in L_k_minus_1:
            return True

    return False
```

✓ 0.0s

- Hàm apriori_gen:

```
def apriori_gen(L_k_minus_1):
    c_k = {}

    keys_list = sorted(list(L_k_minus_1.keys()))
    n = len(keys_list)

    for i in range(n):
        l1 = keys_list[i]
        for j in range(i + 1, n):
            l2 = keys_list[j]

            if l1[:-1] == l2[:-1] and l1[-1] < l2[-1]:
                c = tuple(sorted(set(l1 + l2)))

                if not has_infrequent_subset(c, set(keys_list)):
                    tid_list_c = L_k_minus_1[l1].intersection(L_k_minus_1[l2])
                    c_k[c] = tid_list_c

    return c_k
```

✓ 0.0s

- Hàm vertical_apriori, trong hàm tự code này em trả về itemset và số lượng xuất hiện của nó:

```

def vertical_apriori(D, min_count):
    L1_temp = defaultdict(set)

    for _, row in D.iterrows():
        tid = row['TID']
        items = [item.strip() for item in row['Itemset'].split(',')]

        for item in items:
            L1_temp[(item,)].add(tid)

    L1 = {itemset: tids for itemset, tids in L1_temp.items() if len(tids) >= min_count}

    L = {1: L1}
    k = 2

    while len(L[k - 1]) > 0:
        C_k_with_tids = apriori_gen(L[k - 1])
        L_k = {}

        for c, tid_list in C_k_with_tids.items():
            if len(tid_list) >= min_count:
                L_k[c] = tid_list

        if len(L_k) == 0:
            break

        L[k] = L_k
        k += 1

    L_final = {}
    for _, L_k in L.items():
        for itemset, tid_list in L_k.items():
            L_final[itemset] = len(tid_list)

    return L_final

```

✓ 0.0s

- Chạy thuật toán Vertical apriori với min_count = 4:

```

result = vertical_apriori(df, min_count=4)
for itemset, support in result.items():
    print(f"{itemset}: {support}")

```

✓ 0.0s

```

('Wine',): 16
('Chips',): 14
('Bread',): 16
('Butter',): 15
('Milk',): 17
('Apple',): 15
('Apple', 'Bread'): 12
('Apple', 'Butter'): 11
('Apple', 'Chips'): 10
('Apple', 'Milk'): 11
('Apple', 'Wine'): 11
('Bread', 'Butter'): 13
('Bread', 'Chips'): 9
('Bread', 'Milk'): 13
('Bread', 'Wine'): 13
('Butter', 'Chips'): 9
('Butter', 'Milk'): 13
('Butter', 'Wine'): 11
('Chips', 'Milk'): 10
('Chips', 'Wine'): 9
('Milk', 'Wine'): 14
('Apple', 'Bread', 'Butter'): 9
('Apple', 'Bread', 'Chips'): 8
('Apple', 'Bread', 'Milk'): 9
('Apple', 'Bread', 'Wine'): 10
...
('Apple', 'Bread', 'Chips', 'Milk', 'Wine'): 5
('Apple', 'Butter', 'Chips', 'Milk', 'Wine'): 4
('Bread', 'Butter', 'Chips', 'Milk', 'Wine'): 5
('Apple', 'Bread', 'Butter', 'Chips', 'Milk', 'Wine'): 4

```

- Tạo 1 dataset (new_df) mới có thêm cột Support và thêm điều kiện với min_support:

```

new_results = []

for itemset, count in result.items():
    new_columns = {
        'Itemset': ", ".join(itemset),
        'Count': count,
        'Item_count': len(itemset),
        'Support': (count / df.shape[0])
    }
    new_results.append(new_columns)

new_df = pd.DataFrame(new_results)

min_support = 0.6
new_df = new_df[new_df['Support'] > min_support]
new_df

```

	Itemset	Count	Item_count	Support
0	Wine	16	1	0.727273
1	Chips	14	1	0.636364
2	Bread	16	1	0.727273
3	Butter	15	1	0.681818
4	Milk	17	1	0.772727
5	Apple	15	1	0.681818
20	Milk, Wine	14	2	0.636364

- Tạo support_dict với các item để dễ dàng lấy ra độ support của item đó:

```

df_1_item = new_df[new_df['Item_count'] == 1]
support_dict = dict(zip(df_1_item['Itemset'], df_1_item['Support']))
support_dict

```

	Itemset	Count	Item_count	Support
20	Milk, Wine	14	2	0.636364

- Tạo 1 dataset rules tính toán các luật trong đó dựa vào new_df:

```

min_confidence = 0.8
rules = []
for i in range(new_df.shape[0]):
    if new_df.iloc[i, 2] == 2:
        string = new_df.iloc[i, 0]
        item_A = [item.strip() for item in string.split(',')][0]
        item_B = [item.strip() for item in string.split(',')][1]

        conf_A_B = new_df.iloc[i, 3] / support_dict[item_A]
        if conf_A_B >= min_confidence:
            rules.append([item_A, item_B, float(new_df.iloc[i, 3]), float(conf_A_B)])

        conf_B_A = new_df.iloc[i, 3] / support_dict[item_B]
        if conf_B_A >= min_confidence:
            rules.append([item_B, item_A, float(new_df.iloc[i, 3]), float(conf_B_A)])

rules = pd.DataFrame(rules, columns=['Antecedents', 'Consequents', 'Support', 'Confidence'])
print(rules)

```

✓ 0.0s

	Antecedents	Consequents	Support	Confidence
0	Milk	Wine	0.636364	0.823529
1	Wine	Milk	0.636364	0.875000

So sánh kết quả tự code và code mẫu:

	Item	Support
0	Milk & Wine	0.636364

	Antecedents	Consequents	Support	Confidence
0	Milk	Wine	0.636364	0.823529
1	Wine	Milk	0.636364	0.875000

⇒ Giống nhau